

**The George Washington University**  
Department of Statistics

STAT 4197/6197

Week 3: Working with Formats and Creating New Variables

Detailed Topics:

1. Formats
  - a. Creating, Storing, Accessing, and Maintaining Formats
  - b. Grouping Data Values Using Formats
  - c. Removing Formats and Labels from SAS Data Sets
2. Picture Formats
3. Creating New Variables
  - a. IF-THEN /ELSE IF/ELSE Statements
  - b. SELECT/WHEN/OTHERWISE Statements
  - c. SUM and RETAIN Statements

## LABEL Statement

The LABEL option enforces variables' labels as column headings.

```
proc print data=sashelp.shoes noobs label;
```

The SPLIT= option specifies the split character, which controls line breaks in column headers and implies the use of labels.

```
proc print data=sashelp.shoes noobs split='*';
```

33

## FORMAT Statement

The *FORMAT statement* associates formats to variable values.

```
format inventory sales dollar14.2;
```

Partial Output

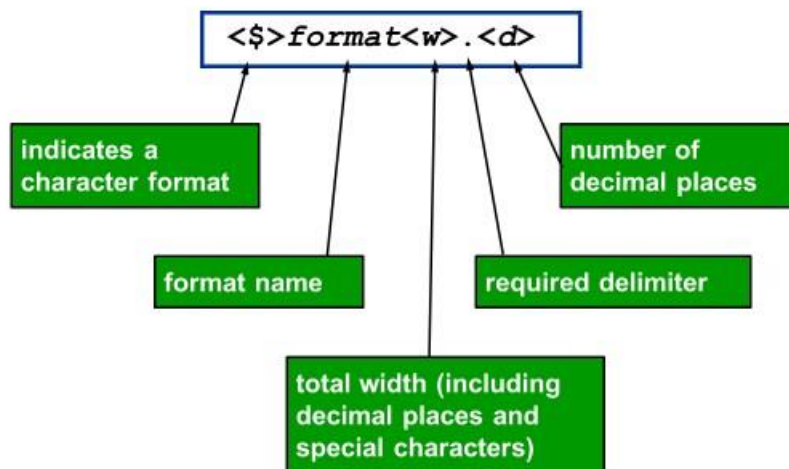
| ----- Region=Asia ----- |            |         |                    |                |
|-------------------------|------------|---------|--------------------|----------------|
| Obs                     | Subsidiary | Product | Total<br>Inventory | Total<br>Sales |
| 57                      | Bangkok    | Boot    | \$9,576.00         | \$1,996.00     |
| 59                      | Bangkok    | Sandal  | \$15,087.00        | \$3,230.00     |
| 62                      | Seoul      | Boot    | \$160,589.00       | \$60,712.00    |
| 65                      | Seoul      | Sandal  | \$21,483.00        | \$4,978.00     |
| -----                   |            |         |                    |                |
| Region                  |            |         | \$206,735.00       | \$70,916.00    |

A *format* is an instruction that SAS uses to write data values.

34

## FORMAT Statement

Formats have the following form:



35

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## FORMAT Statement

| Stored Value | Format    | Displayed Value |
|--------------|-----------|-----------------|
| Washington   | \$4.      | Wash            |
| 1234.4567    | 8.0       | 1234            |
| 1234.4567    | 8.2       | 1234.46         |
| 1234.4567    | comma8.2  | 1,234.46        |
| 1234.4567    | dollar9.2 | \$1,234.46      |

36



## FORMAT Statement

Partial  
Output

| ----- Region=Western Europe ----- |         |                    |                |
|-----------------------------------|---------|--------------------|----------------|
| Subsidiary                        | Product | Total<br>Inventory | Total<br>Sales |
| Copenhagen                        | Boot    | \$4,657.00         | \$1,663.00     |
| Geneva                            | Boot    | \$171,030.00       | \$41,341.00    |
|                                   | dal     | \$3,529.00         | \$736.00       |
|                                   | t       | \$301,779.00       | \$65,610.00    |
|                                   | dal     | \$4,618.00         | \$977.00       |
|                                   | t       | \$341,911.00       | \$76,349.00    |
|                                   | dal     | \$24,253.00        | \$1,650.00     |
|                                   | t       | \$289,527.00       | \$54,449.00    |
|                                   | dal     | \$11,111.00        | \$5,217.00     |
| Madrid                            | Boot    | \$1,027.00         | \$1,179.00     |
| Paris                             | Boot    | \$41,506.00        | \$19,196.00    |
| Paris                             | Sandal  | \$23,816.00        | \$1,520.00     |
| Rome                              | Boot    | \$209,271.00       | \$36,244.00    |
| Rome                              | Sandal  | \$4,611.00         | \$1,249.00     |
| -----                             |         |                    |                |
| Region                            |         | \$1,432,646.00     | \$307,380.00   |
|                                   |         | =====              | =====          |
|                                   |         | \$12,956,946.00    | \$3,218,979.00 |

What minimum widths are needed to complete the FORMAT statement for this desired output?

```
format inventory dollar14.2 sales dollar13.2;
```

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Business Scenario

Display country names instead of country codes in a report.

Current Report (partial output)

| Obs | Employee_ID | Salary    | Country | Birth_Date | Hire_Date |
|-----|-------------|-----------|---------|------------|-----------|
| 1   | 120102      | \$108,255 | AU      | AUG1973    | JUN1993   |
| 2   | 120103      | \$87,975  | AU      | JAN1953    | JAN1978   |
| 3   | 120121      | \$26,600  | AU      | AUG1948    | JAN1978   |



Desired Report (partial output)

| Obs | Employee_ID | Salary    | Country   | Birth_Date | Hire_Date |
|-----|-------------|-----------|-----------|------------|-----------|
| 1   | 120102      | \$108,255 | Australia | AUG1973    | JUN1993   |
| 2   | 120103      | \$87,975  | Australia | JAN1953    | JAN1978   |
| 3   | 120121      | \$26,600  | Australia | AUG1948    | JAN1978   |

## Business Scenario

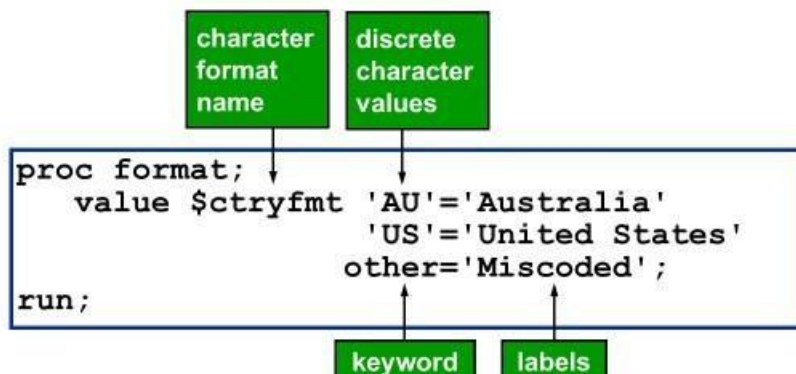
Management has requested that country names, instead of country codes, be used in reports.

| Country | Population | Country_ID |
|---------|------------|------------|
| AU      | 20,000,000 | 160        |
| CA      | .          | 260        |
| DE      | 80,000,000 | 394        |
| IL      | 5,000,000  | 475        |



| Country   | Population | Country_ID |
|-----------|------------|------------|
| Australia | 20,000,000 | 160        |
| Canada    | .          | 260        |
| Germany   | 80,000,000 | 394        |
| Israel    | 5,000,000  | 475        |

## Defining a Character Format

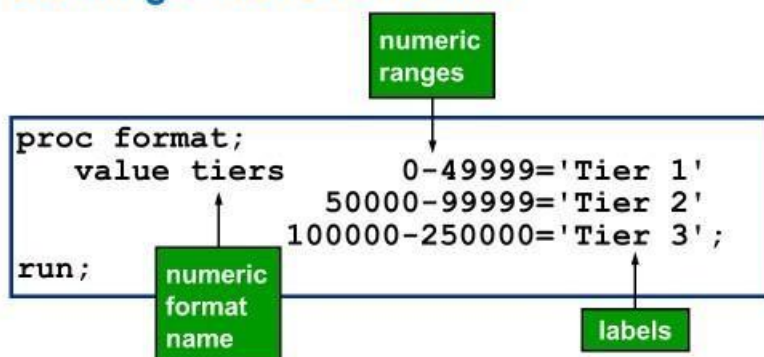


The OTHER keyword includes all values that do not match any other value or range.

26

p105d03

## Defining a Numeric Format



37

p105d04

The FORMAT procedure enables you to define your own formats for variable values. Formats determine how variable values are printed.

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## VALUE Statement

```
VALUE format-name range1='label '  
                  range2='label '  
                  ... ;
```

A format name

- can be up to 32 characters in length
- for character formats, must begin with a dollar sign (\$), followed by a letter or underscore
- for numeric formats, must begin with a letter or underscore
- cannot end in a number
- cannot be given the name of a SAS format
- cannot include a period in the VALUE statement.

22



## Applying a Format

User-defined and SAS formats can be applied in a single FORMAT statement.

```
proc print data=orion.sales label;
  var Employee_ID Job_Title Salary
      Country Birth_Date Hire_Date;
  format Salary dollar10.
         Birth_Date Hire_Date monyy7.
         Country $ctryfmt.;
run;
```

 A period (for example, at the end of the \$CTRYFMT format) is required when user-defined formats are used in a FORMAT statement.

27

p105d03

## Viewing the Output

Partial PROC PRINT Output

| Obs | Employee_ID | Salary    | Country   | Birth_Date | Hire_Date |
|-----|-------------|-----------|-----------|------------|-----------|
| 1   | 120102      | \$108,255 | Australia | AUG1973    | JUN1993   |
| 2   | 120103      | \$87,975  | Australia | JAN1953    | JAN1978   |
| 3   | 120121      | \$26,600  | Australia | AUG1948    | JAN1978   |
| 4   | 120122      | \$27,475  | Australia | JUL1958    | JUL1982   |
| 5   | 120123      | \$26,190  | Australia | SEP1968    | OCT1989   |

21

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## VALUE Statement

```
VALUE format-name range1='label '
                    range2='label '
                    ...;
```

Each range can be

- a single value
- a range of values
- a list of values.

Labels

- can be up to 32,767 characters in length
- are enclosed in quotation marks.

23

## Single Values

On the left side of the equal sign, you can have single values. Character values should be enclosed in quotation marks.

```
proc format;
  value $gender 'F' = 'Female'
                'M' = 'Male'
                other = 'Miscoded';
  value wtrange low - <100 = 'Under 100 lbs'
                100 - high = '100+ lbs';
run;
```

case-sensitive

You can have multiple single values with commas separating the values.

```
value $gender 'F', 'FEM', 'FEMALE' = 'Female'
              'M', 'MAL', 'MALE'   = 'Male';
```

61

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Ranges

On the left side of the equal sign, you can have ranges.

```
proc format;  
  value $gender 'F' = 'Female'  
               'M' = 'Male'  
               other = 'Miscoded';  
  value wtrange low - <100 = 'Under 100 lbs'  
               100 - high = '100+ lbs';  
run;
```

For character ranges, each string should be enclosed in quotation marks (example: 'A' - 'Z').

## User-Defined Format Example

Ranges can be specified using lists, ranges, discrete values, and keywords.

```
proc format;
  value mnthfmt 1,2,3='Qtr 1'
                4-6='Qtr 2'
                7-9='Qtr 3'
                10-12='Qtr 4'
                .='missing'
                other='unknown';
run;
```

37

## Keywords

- OTHER matches all values that do not match any other value or range.
- LOW encompasses lowest possible value.  
LOW does not include missing for numeric variables.  
LOW does include missing for character variables.
- HIGH encompasses highest possible value.
- LOW - HIGH encompasses all values.

```
proc format;
  value $gender 'F' = 'Female'
                'M' = 'Male'
                other = 'Miscoded';
  value wtrange low - <100 = 'Under 100 lbs'
                100 - high = '100+ lbs';
run;
```

70

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Multiple User-Defined Formats

Multiple VALUE statements can be included in a single PROC FORMAT step.

```
proc format;
  value $ctryfmt  'AU'='Australia'
                  'US'='United States'
                  other='Miscoded';

  value tiers     low-<50000  ='Tier 1'
                  50000-<100000='Tier 2'
                  100000-high  ='Tier 3';
run;
```

38

p105d07

## Nesting Formats

In the VALUE statement, you can specify that the format use a second format as the formatted value.

Enclose the format name in square brackets:

```
proc format library=orion.MyFmts;
  value $extra  ' '='Unknown'
               other=[$country30.];
run;
```

value=[existing-format]

18

p210d01

## Applying a Numeric Format

Part 1

```
proc format;
  value tiers      low-<50000 ='Tier 1'
                    50000-<100000='Tier 2'
                    100000-high  ='Tier 3';
run;
```

Part 2

```
proc print data=orion.sales;
  var Employee_ID Job_Title Salary
      Country Birth_Date Hire_Date;
  format Birth_Date Hire_Date monyy7.
         Salary tiers.;
run;
```

35

p105d06

## Viewing the Output

Partial PROC PRINT Output

| Obs | Employee_ID | Job_Title     | Salary | Country | Birth_Date | Hire_Date |
|-----|-------------|---------------|--------|---------|------------|-----------|
| 1   | 120102      | Sales Manager | Tier 3 | AU      | AUG1973    | JUN1993   |
| 2   | 120103      | Sales Manager | Tier 2 | AU      | JAN1953    | JAN1978   |
| 3   | 120121      | Sales Rep. II | Tier 1 | AU      | AUG1948    | JAN1978   |
| 4   | 120122      | Sales Rep. II | Tier 1 | AU      | JUL1958    | JUL1982   |
| 5   | 120123      | Sales Rep. I  | Tier 1 | AU      | SEP1968    | OCT1989   |

36

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

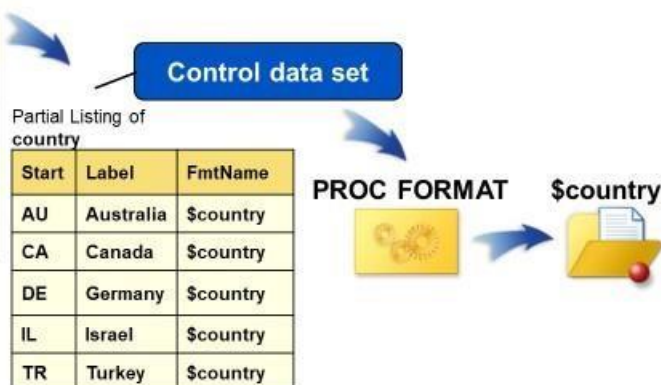


## Using a Control Data Set to Create a Format

Instead of entering all of the values in PROC FORMAT, you can create a format from a SAS data set that contains the code and value information.

Partial Listing of  
orion.country

| Country | Country_Name |
|---------|--------------|
| AU      | Australia    |
| CA      | Canada       |
| DE      | Germany      |
| IL      | Israel       |
| TR      | Turkey       |



## Using a Control Data Set to Create a Format

Partial **orion.country**

| Country | Country_Name | Population | Country_ID | Continent_ID | Country_FormerName |
|---------|--------------|------------|------------|--------------|--------------------|
| AU      | Australia    | 20,000,000 | 160        | 96           |                    |
| CA      | Canada       | .          | 260        | 91           |                    |
| DE      | Germany      | 80,000,000 | 394        | 93           | East/West Germany  |
| IL      | Israel       | 5,000,000  | 475        | 95           |                    |

DATA Step



Start

Label

FmtName

```
data country;
  keep Start Label FmtName;
  retain FmtName '$country';
  set orion.country(rename=(Country=Start
                           Country_Name=Label));
run;
```

6

## Using a Control Data Set to Create a Format

Use the CNTLIN= option to read the data and create the format.

```
proc format cntlin=country;
run;
```

CNTLIN=SAS-data-set



The variables **FmtName**, **Start**, and **Label** are required in order to create a format from a CNTLIN data set.

7

Use CNTLIN option with PROC FORMAT to specify a SAS data set for creating formats/informats.  
Use CNTLOUT option with PROC FORMAT to generate a SAS data set with information about formats/informats.

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.



## Where Formats Are Stored

Without the LIBRARY= option, formats are stored in the **work.formats** catalog and exist for the duration of the SAS session.

```
PROC FORMAT;
```

If the LIBRARY= option specifies only a *libref*, formats are stored permanently in ***libref.formats***.

```
PROC FORMAT LIBRARY=libref;
```

If the LIBRARY= option specifies *libref.catalog*, formats are stored permanently in that catalog.

```
PROC FORMAT LIBRARY=libref.catalog;
```

15

- User-written formats can be stored in catalogs.
- The storage location for formats/catalogs can be either temporary or permanent.

## Using the LIBRARY= Option

If the LIBRARY= option specifies only a libref, formats are stored permanently in a catalog named **formats**, referenced by **libref.formats**.

```
proc format library=orion;
```

```
PROC FORMAT LIBRARY=libref;
```



**orion.formats**



13

## Using the LIBRARY= Option

If the LIBRARY= option specifies *libref.catalog*, formats are stored permanently in that catalog.

```
proc format library=orion.MyFmts;
```

```
PROC FORMAT LIBRARY=libref.catalog;
```



**orion.MyFmts**

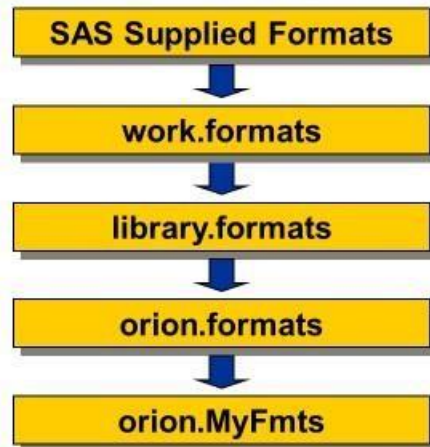


Store frequently used formats in permanent catalogs.

14

## Using the FMTSEARCH= System Option

```
options fmtsearch=(orion orion.MyFmts) ;
```



## CATALOG Procedure

The CATALOG procedure manages entries in SAS catalogs.

```
proc catalog cat=orion.MyFmts;
  contents;
quit;
```

```
PROC CATALOG CATALOG=ilbref.catalog;
  CONTENTS;
QUIT;
```

| Contents of Catalog ORION.MYFMTS |              |         |                  |                  |
|----------------------------------|--------------|---------|------------------|------------------|
| #                                | Name         | Type    | Create Date      | Modified Date    |
| 1                                | DATES        | FORMAT  | 29Jan08:16:26:39 | 29Jan08:16:26:39 |
| 2                                | COUNTRY      | FORMATC | 29Jan08:16:33:30 | 29Jan08:16:33:30 |
| 3                                | COUNTRY_NAME | FORMATC | 20Apr09:15:30:14 | 20Apr09:15:30:14 |
| 4                                | EXTRA        | FORMATC | 20Apr09:15:30:14 | 20Apr09:15:30:14 |

16

p210d01

## Documenting Formats

You can use the FMTLIB option in the PROC FORMAT statement to document the format.

```
proc format library=orion.MyFmts fmtlib;
  select $country;
run;
```

| FORMAT NAME: \$COUNTRY LENGTH: 13 NUMBER OF VALUES: 7  |     |                                       |
|--------------------------------------------------------|-----|---------------------------------------|
| MIN LENGTH: 1 MAX LENGTH: 40 DEFAULT LENGTH 13 FUZZ: 0 |     |                                       |
| START                                                  | END | LABEL (VER. V7 V8 05MAY2009:12:34:42) |
| AU                                                     | AU  | Australia                             |
| CA                                                     | CA  | Canada                                |
| DE                                                     | DE  | Germany                               |
| IL                                                     | IL  | Israel                                |
| TR                                                     | TR  | Turkey                                |
| US                                                     | US  | United States                         |
| ZA                                                     | ZA  | South Africa                          |

17

p210d01

## Picture Statement in the FORMAT Procedure

| With the picture statement, you can                                                                                                                                                                                                                                                                                 | Options with the Picture Statement                                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Display numbers with leading zeros</li> <li>• Fill numbers with special characters</li> <li>• Insert message characters into numbers.</li> <li>• Customizing a date, time, or datetime display</li> <li>• Format currency when there is no SAS format available</li> </ul> | <p>NOEDIT= Specify that numbers are message characters rather than digit selectors</p> <p>PREFIX= Specify a character prefix for the formatted value</p> <p>FILL= Specify a character that completes the formatted value</p> <p>MULT= Specify a number to multiply the variable's value by before it is formatted</p> <p>Source: SAS® Documentation</p> |

## Creating New Variables using the Assignment Statement



### Completed Program

Place the assignment statement in the DATA step.

```
data pilotdata;
  infile "&path\pilot.dat";
  input EmployeeID $ 1-6
        FirstName $ 7-19
        LastName $ 20-34
        JobCode $ 35-41
        Salary 42-47
        Category $ 48-50;
  Bonus=Salary*0.10;
run;
```

## Assignment Statement

The assignment statement evaluates an expression and stores the result in a new variable or an existing variable.

Examples:

```
name = 'Jane Doe';
revenue = 157900;
date = '10MAY2007'd;
total = price * quantity;
cityst = city !! state;
product = upcase(product);
average = mean(jan, feb, mar);
```

variable

expression

Assignment statements evaluate the expression on the right side of the equal sign and store the result in the variable that is specified on the left side of the equal sign.

3

## Expression

The *expression* is a sequence of operands and operators that form a set of instructions that produce a value.

- *Operands* are
  - constants (character or numeric)
  - variables (character or numeric).
- *Operators* are
  - symbols that represent an arithmetic calculation or concatenation
  - a SAS function.

6



## Operands

A *constant* is a number or a character string that indicates a fixed value.

```
name = 'Jane Doe';
revenue = 157900;
date = '10MAY2007'd;
```

Character constants must be enclosed in quotation marks.

Character constants are enclosed in quotation marks, but names of variables are not enclosed in quotation marks.

```
total = price * quantity;
cityst = city !! state;
product = upcase(product);
average = mean(jan, feb, mar);
```

7

## Operators

- Arithmetic operators indicate that an arithmetic calculation is performed.
- A concatenation operator concatenates character values.
- A SAS function performs a computation or system manipulation on arguments and returns a value.

```
total = price * quantity;
cityst = city !! state;
product = upcase(product);
average = mean(jan, feb, mar);
```

8



## Arithmetic Operators

Possible arithmetic operators:

| Symbol | Definition     |
|--------|----------------|
| **     | exponentiation |
| *      | multiplication |
| /      | division       |
| +      | addition       |
| -      | subtraction    |

- If a missing value is an operand for an arithmetic operator, the result is a missing value.

## Determine the Expression

Expressions can contain **operators** and **parentheses**.

| Symbol | Definition     | Example           |
|--------|----------------|-------------------|
| +      | Addition       | CurrentRate+10.27 |
| -      | Subtraction    | gross_pay-tax     |
| *      | Multiplication | price*quantity    |
| /      | Division       | Minutes/60        |
| ( )    | Grouping       | (jan+feb+mar)/3   |
| **     | Exponentiation | x**2              |

20

## Example

```
data newprice;
  set golf.supplies;
  saleprice = price * 0.75;
  saletype = '25% off';
  format price saleprice dollar8.2;
run;
```

| VIEWTABLE: Golf.Supplies |        |          |       |
|--------------------------|--------|----------|-------|
|                          | mfg    | type     | price |
| 1                        | Crew   | Distance | 8.1   |
| 2                        | Crew   | Spin     | 8.25  |
| 3                        | Crew   | Titanium | 9.5   |
| 4                        | Hi-fly | X12000   | 13.75 |
| 5                        | Hi-fly | X22000   | 14.6  |
| 6                        | White  | Strata   | 10.6  |
| 7                        | White  | Aero     | 12.3  |
| 8                        | White  | XL       | 14.5  |
| 9                        | White  | Flite    | 16.2  |

| VIEWTABLE: Work.Newprice |        |          |         |           |          |
|--------------------------|--------|----------|---------|-----------|----------|
|                          | mfg    | type     | price   | saleprice | saletype |
| 1                        | Crew   | Distance | \$8.10  | \$6.08    | 25% off  |
| 2                        | Crew   | Spin     | \$8.25  | \$6.19    | 25% off  |
| 3                        | Crew   | Titanium | \$9.50  | \$7.13    | 25% off  |
| 4                        | Hi-fly | X12000   | \$13.75 | \$10.31   | 25% off  |
| 5                        | Hi-fly | X22000   | \$14.60 | \$10.95   | 25% off  |
| 6                        | White  | Strata   | \$10.60 | \$7.95    | 25% off  |
| 7                        | White  | Aero     | \$12.30 | \$9.23    | 25% off  |
| 8                        | White  | XL       | \$14.50 | \$10.88   | 25% off  |
| 9                        | White  | Flite    | \$16.20 | \$12.15   | 25% off  |

10

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Example

```
data newprice;
  set golf.supplies;
  ➔ saleprice = price * 0.75;
  ➔ saletype = '25% off';
  format price saleprice dollar8.2;
run;
```

Which statement is **true** concerning the new variables?

- A. **saleprice** and **saletype** are numeric (8 bytes).
- B. **saleprice** and **saletype** are character (7 bytes).
- ☒ C. **saleprice** is numeric (8 bytes) and **saletype** is character (7 bytes).
- D. **saleprice** is numeric (8 bytes) and **saletype** is character (8 bytes).

12

## IF-THEN / ELSE Statements

- The IF-THEN statement executes **a statement** for observations that meet specific conditions.
- The optional ELSE statement gives an alternative action if the THEN clause is not executed.

```
data newprice;
  set golf.supplies;
  ➔ if mfg='Crew' then saleprice=price*0.75;
  ➔ else if mfg='Hi-fly' then saleprice=price*0.70;
  ➔ else if mfg='White' then saleprice=price*0.90;
  format price saleprice dollar8.2;
run;
```

22

## ELSE Statements

```
if mfg='Crew' then saleprice=price*0.75;
else if mfg='Hi-fly' then saleprice=price*0.70;
else if mfg='White' then saleprice=price*0.90;
```

- Using IF-THEN statements **without** the ELSE statement causes SAS to evaluate all IF-THEN statements.
- Using IF-THEN statements **with** the ELSE statement causes SAS to execute the IF-THEN statements until SAS encounters the first true statement. Subsequent IF-THEN statements are not evaluated.

29

## ELSE Statements

```
if mfg='Crew' then saleprice=price*0.75;
else if mfg='Hi-fly' then saleprice=price*0.70;
else saleprice=price*0.90;
```

The final ELSE statement can be coded without an IF-THEN statement to direct all previous false conditions into the final condition.

32

Conditional statements (e.g., IF-THEN /ELSE IF logic) can create values for a new variable based on whether a condition is true or false.

## Completed Program



The correct approach for our program is to use IF-THEN/ELSE IF logic.

```
data pilotdata;
  infile "&path\pilot.dat";
  input EmployeeID $ 1-6
        FirstName $ 7-19
        LastName $ 20-34
        JobCode $ 35-41
        Salary 42-47
        Category $ 48-50;
  Bonus=Salary*0.10;
  if JobCode='PILOT1' then
    NewSalary=Salary*1.05;
  else if JobCode='PILOT2' then
    NewSalary=Salary*1.07;
  else if JobCode='PILOT3' then
    NewSalary=Salary*1.09;
run;
```

42

in10d04

If you use a variable from the input data source as part of your expression (as we did with the JobCode and Salary Variables), the conditional logic statement must be placed after the INFILE and INPUT statements.



## Table Lookup Techniques

The technique that is used to perform a table lookup depends on the data.



| Table Location | Technique                                         | Lookup Table               |
|----------------|---------------------------------------------------|----------------------------|
| DATA step      | IF-THEN/ELSE statements                           | SAS programming statements |
| Disk           | SQL join<br>merge<br>SET/SET KEY=                 | SAS data set               |
| Memory         | FORMAT statement<br>PUT statement<br>PUT function | user-defined format        |
|                | array reference                                   | array                      |
|                | FIND method                                       | hash object                |

12

## IF-THEN/ELSE Statements

Lookup tables can be SAS programming statements.



```
data countryinfo;
  set orion.country;
  if ContinentID=91
    then Continent='North America';
  else if ContinentID=93
    then Continent='Europe';
  else if ContinentID=94
    then Continent='Africa';
  else if ContinentID=95
    then Continent='Asia';
  else if ContinentID=96
    then Continent='Australia/Pacific';
run;
```

7

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.



## Creating Lookup Values (i.e., New Variables) with the DO statement



### IF-THEN DO / ELSE DO Statements

```

data newprice;
  infile 'raw-data-file';
  input mfg $ type $ price;
  length saletype $ 18;
  if mfg='Crew' then do;
    pct=0.75;
    saleprice = price * pct;
    saletype = '25% off';
  end;
  else if mfg='Hi-fly' then do;
    pct=0.70;
    saleprice = price * pct;
    saletype = '30% off';
  end;
  else do;
    pct=0.90;
    saleprice = price * pct;
    saletype = '10% Storewide Sale';
  end;
  format price saleprice dollar8.2;
run;

```

1

2

3

Conditional statements can be used in DATA steps that read raw data files or data sets.

36



### IF-THEN DO / ELSE DO Statements

```

data newprice;
  infile 'raw-data-file';
  input mfg $ type $ price;
  length saletype $ 18;
  if mfg='Crew' then do;
    pct=0.75;
    saleprice = price * pct;
    saletype = '25% off';
  end;
  else if mfg='Hi-fly' then do;
    pct=0.70;
    saleprice = price * pct;
    saletype = '30% off';
  end;
  else do;
    pct=0.90;
    saleprice = price * pct;
    saletype = '10% Storewide Sale';
  end;
  format price saleprice dollar8.2;
run;

```

A LENGTH statement controls the byte size of saletype. Without the statement, byte size would be 7 (25% off).

Three variables are being created. pct and saleprice are numeric (8 bytes). saletype is character (18 bytes).

37

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Creating Variables Using PUT Function and Formats

### Overview of a Format

```
proc format;
  value Cont Name
    91='North America'
    93='Europe'
    94='Africa'
    95='Asia'
    96='Australia/Pacific';
run;

data country_info;
  set orion.country;
  Continent=put(Continent_ID,Cont_Name.);
run;
```

The FORMAT step compiles the format and stores it on disk.

When the PUT function executes, the format is loaded into memory, and a binary search is used to retrieve the format value.

p304d03

35

### User-Defined Formats

Lookup tables can be user-defined formats that are accessed with a FORMAT statement or PUT function.

```
proc format;
  value ContName
    91='North America' 93='Europe'
    94='Africa'         95='Asia'
    96='Australia/Pacific';
run;

proc print data=orion.country;
  format ContinentID ContName.;
run;

data countryinfo;
  set orion.country;
  Continent=put(ContinentID,ContName.);
run;
```



10

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## SELECT / WHEN / OTHERWISE Statements

An alternative to IF-THEN statements is SELECT / WHEN / OTHERWISE statements.

- The SELECT statement begins a SELECT group.
- SELECT groups contain WHEN statements that identify SAS statements that are executed when a particular condition is true.
- A SELECT group must use at least one WHEN statement.
- An optional OTHERWISE statement specifies a statement to be executed if no WHEN condition is met.
- An END statement ends a SELECT group.

41

## SELECT / WHEN / OTHERWISE Statements

IF  
THEN  
ELSE

```
data newprice;
  set golf.supplies;
  if mfg='Crew' then
    saleprice=price*0.75;
  else if mfg='Hi-fly' then
    saleprice=price*0.70;
  else if mfg='White' then
    saleprice=price*0.90;
  format price saleprice dollar8.2;
run;
```

SELECT  
group

```
data newprice;
  set golf.supplies;
  select(mfg);
    when('Crew') saleprice=price*0.75;
    when('Hi-fly') saleprice=price*0.70;
    when('White') saleprice=price*0.90;
  end;
  format price saleprice dollar8.2;
run;
```

42

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.



## SELECT / WHEN / OTHERWISE Statements

```
select(mfg) ;
  when('Crew') saleprice=price*0.75;
  when('Hi-fly') saleprice=price*0.70;
  when('White') saleprice=price*0.90;
end;
```

Based on the partial program above, what is the result if an observation has a value of **mfg='X-treme'** ?

```
ERROR: Unsatisfied WHEN clause and no OTHERWISE clause at line
        618 column 3.
mfg=X-treme type=Strata price=$10.60 saleprice=. _ERROR_=1
_N=10
NOTE: The SAS System stopped processing this step because of
      errors.
```

44

## SELECT / WHEN / OTHERWISE Statements

A null OTHERWISE statement prevents SAS from issuing an error message when all WHEN conditions are false.

```
select(mfg) ;
  when('Crew') saleprice=price*0.75;
  when('Hi-fly') saleprice=price*0.70;
  when('White') saleprice=price*0.90;
end;
```

```
ERROR: Unsatisfied WHEN clause and no
OTHERWISE clause at line 618 column 3.
mfg=X-treme type=Strata price=$10.60
saleprice=. _ERROR_=1 _N=10
```

```
select(mfg) ;
  when('Crew') saleprice=price*0.75;
  when('Hi-fly') saleprice=price*0.70;
  when('White') saleprice=price*0.90;
  otherwise;
end;
```

no ERROR

45

sas THE POWER TO EXCEL

SELECT group

```

data newprice;
infile 'raw-data-file';
input mfg $ type $ price;
length saletype $ 18;
select(mfg) ;
  1 when('Crew') do;
    pct=0.75;
    saleprice = price * pct;
    saletype = '25% off';
  end;
  2 when('Hi-fly') do;
    pct=0.70;
    saleprice = price * pct;
    saletype = '30% off';
  end;
  3 otherwise do;
    pct=0.90;
    saleprice = price * pct;
    saletype = '10% Storewide Sale';
  end;
end;
format price saleprice dollar8.2;
run;

```

All previous false conditions fall into the final condition.

47

sas THE POWER TO EXCEL

## IF-THEN DO / ELSE DO Statements

```

data newprice;
infile 'raw-data-file';
input mfg $ type $ price;
length saletype $ 18;
if mfg='Crew' then do;
  pct=0.75;
  saleprice = price * pct;
  saletype = '25% off';
end;
else if mfg='Hi-fly' then do;
  pct=0.70;
  saleprice = price * pct;
  saletype = '30% off';
end;
else do;
  pct=0.90;
  saleprice = price * pct;
  saletype = '10% Storewide Sale';
end;
format price saleprice dollar8.2;
run;

```

All previous false conditions fall into the final condition.

38

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## SELECT / WHEN / OTHERWISE Statements

Additional examples:

```
select (payclass) ;
  when ('monthly') amt=salary;
  when ('hourly')  amt=hrlywage*40;
  otherwise;
end;
```

```
select;
  when (12000<=revenue<=24000) target='goal';
  when (revenue<12000) target='below';
  when (revenue>24000) target='above';
  otherwise;
end;
```

```
select;
  when (mon in ('JUN', 'JUL', 'AUG') and temp>70)
    status='SUMMER';
  when (mon in ('MAR', 'APR', 'MAY'))
    status='SPRING';
  otherwise status='FALL OR WINTER';
end;
```

48

You can replace the IF/THEN logic by using the PUT function with a custom format in a assignment statement.

## Business Scenario

A manager has asked to see daily sales for April, as well as a month-to-date total for each day for her department.

Partial

**orion.aprsales**

| SaleDate  | SaleAmt |
|-----------|---------|
| 01APR2011 | 498.49  |
| 02APR2011 | 946.50  |
| 03APR2011 | 994.97  |
| 04APR2011 | 564.59  |
| 05APR2011 | 783.01  |
| 06APR2011 | 228.82  |
| 07APR2011 | 930.57  |



Partial **mnthtot**

| SaleDate  | Sale Amt | Mth2Dte   |
|-----------|----------|-----------|
| 01APR2011 | 498.49   | 498.49    |
| 02APR2011 | + 946.50 | = 1444.99 |
| 03APR2011 | 994.97   | 2439.96   |
| 04APR2011 | 564.59   | 3004.55   |
| 05APR2011 | 783.01   | 3787.56   |



## 4.01 Short Answer Poll – Correct Answer

Open and submit the program in **pdm04a01**. Does this program create the correct values for **Mth2Dte**?

```
data mnthtot;
  set orion.aprsales;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Partial **orion.aprsales**

| SaleDate  | Sale Amt | Mth2Dte |
|-----------|----------|---------|
| 01APR2011 | 498.49   | .       |
| 02APR2011 | 946.50   | .       |
| 03APR2011 | 994.97   | .       |
| 04APR2011 | 564.59   | .       |

No, the program creates **Mth2Dte** with all missing values.

5

psm04a01

## Creating an Accumulating Variable

By default, variables created with an assignment statement are initialized to missing at the top of each iteration of the DATA step.

```
data mnthtot;
  set orion.aprsales;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Initialized to missing at the top of the DATA step

**Mth2Dte** is an example of an accumulating variable that needs to keep its value from one observation to the next.

6

## Creating an Accumulating Variable

Retain the values of **Mth2Dte** and set an initial value.

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

**RETAIN** *variable-name* <initial-value> ...;



If you do not supply an initial value, all the values of **Mth2Dte** will be missing.

7

psm04d01

## RETAIN Statement: Details

The RETAIN statement

- retains the value of the variable in the PDV across iterations of the DATA step
- initializes the retained variable to missing or a specified initial value before the first iteration of the DATA step
- is a compile-time-only statement.

 The RETAIN statement has no effect on variables that are read with SET, MERGE, or UPDATE statements. Variables read from SAS data sets are retained automatically.

8

## Compilation: Create an Accumulating Variable

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|-------------------------------------------------------------------------------------------|
|          |         |                                                                                           |

**Retain flag**

9

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

**Initialize PDV**

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|---------------------------------------------------------------------------------------------|
| .        | .       | 0                                                                                           |



The input SAS data set must be sorted by **SaleDate** for the program to produce the correct results.

10

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|-------------------------------------------------------------------------------------------|
| 18718    | 498.49  | 0                                                                                         |

11

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

0 + 498.49

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|---------------------------------------------------------------------------------------------|
| 18718    | 498.49  | 498.49                                                                                      |

12

...

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.



## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Implicit OUTPUT;  
Implicit RETURN;

PDV

| SaleDate | SaleAmt | Mth2Dte |
|----------|---------|---------|
| 18718    | 498.49  | 498.49  |

Write observation to mnthtot

13

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Mth2Dte is not reinitialized.

PDV

| SaleDate | SaleAmt | Mth2Dte |
|----------|---------|---------|
| 18718    | 498.49  | 498.49  |

14

...



## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|-------------------------------------------------------------------------------------------|
| 18719    | 946.50  | 498.49                                                                                    |

15

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

PDV

| SaleDate | SaleAmt |  Mth2Dte |
|----------|---------|---------------------------------------------------------------------------------------------|
| 18719    | 946.50  | 1444.99                                                                                     |



498.49 + 946.50

16

...

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Implicit OUTPUT;  
Implicit RETURN;

PDV

| SaleDate | SaleAmt | Mth2Dte |
|----------|---------|---------|
| 18719    | 946.50  | 1444.99 |

Write observation to mnthtot

17

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Mth2Dte is not reinitialized.

PDV

| SaleDate | SaleAmt | Mth2Dte |
|----------|---------|---------|
| 18719    | 946.50  | 1444.99 |

18

...

## Execution: Create an Accumulating Variable

| SaleDate | SaleAmt |
|----------|---------|
| 18718    | 498.49  |
| 18719    | 946.50  |
| 18720    | 994.97  |
| 18721    | 564.59  |
| 18722    | 783.01  |

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=Mth2Dte+SaleAmt;
run;
```

Continue until EOF

PDV

| SaleDate | SaleAmt | Mth2Dte |
|----------|---------|---------|
| 18719    | 946.50  | 1444.99 |

19

## Creating an Accumulating Variable

```
proc print data=mnthtot noobs;
  format SaleDate date9.;
run;
```

Partial PROC PRINT Output

| SaleDate  | Sale Amt | Mth2Dte |
|-----------|----------|---------|
| 01APR2011 | 498.49   | 498.49  |
| 02APR2011 | 946.50   | 1444.99 |
| 03APR2011 | 994.97   | 2439.96 |
| 04APR2011 | 564.59   | 3004.55 |
| 05APR2011 | 783.01   | 3787.56 |

20

psm04d01

Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.

## SUM Function

A RETAIN statement along with a SUM function in an assignment statement can be used to create **Mth2Dte**.

```
data mnthtot;
  set orion.aprsales;
  retain Mth2Dte 0;
  Mth2Dte=sum(Mth2Dte,SaleAmt) ;
run;
```

 The SUM function ignores missing values.

25

## Sum Statement

Use the sum statement to create **Mth2Dte**.

```
data mnthtot2;
  set work.aprsales2;
  Mth2Dte+SaleAmt;
run;
```

*variable + expression;*

Specifics about **Mth2Dte**:

- initialized to zero
- automatically retained
- increased by the value of **SaleAmt** for each observation
- ignored missing values of **SaleAmt**

26

psm04d02

## Sum Statement

```
proc print data=mnthtot2 noobs;
  format SaleDate date9.;
run;
```

Partial PROC PRINT Output (30 Total Observations)

| SaleDate  | SaleAmt | Mth2Dte |
|-----------|---------|---------|
| 01APR2011 | 498.49  | 498.49  |
| 02APR2011 | .       | 498.49  |
| 03APR2011 | 994.97  | 1493.46 |
| 04APR2011 | 564.59  | 2058.05 |
| 05APR2011 | 783.01  | 2841.06 |

27

## Accumulator Variables

An *accumulator variable* is a variable that adds on an expression.

Partial Output



| Maine<br>County | Population<br>2000 | Total<br>Population | Total<br>Counties |
|-----------------|--------------------|---------------------|-------------------|
| Androscoggin    | 103,793            | 103,793             | 1                 |
| Aroostook       | 73,938             | 177,731             | 2                 |
| Cumberland      | 265,612            | 443,343             | 3                 |
| Franklin        | 29,467             | 472,810             | 4                 |
| Hancock         | 51,791             | 524,601             | 5                 |
| Kennebec        | 117,114            | 641,715             | 6                 |
| Knox            | 39,618             | 681,333             | 7                 |
| Lincoln         | 33,616             | 714,949             | 8                 |
| Oxford          | 54,755             | 769,704             | 9                 |
| Penobscot       | 144,919            | 914,623             | 10                |

accumulator variables

52

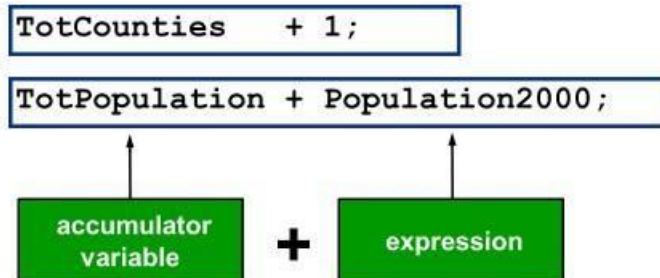
Acknowledgments: Portions of SAS' copyrighted SAS course content are reproduced here with permission of SAS Institute Inc., Cary, NC, USA. SAS Institute Inc. makes no warranties with respect to these materials and disclaims all liability therefor. Neither the GW nor the instructor shall be held liable or responsible to any person or entity with respect to any loss or incidental or consequential damages caused by anyone's use of the information or SAS codes contained herein. Circulating the handouts is strictly forbidden.



## Sum Statement

The *sum statement* adds the result of an expression to an accumulator variable.

Examples:



53

## Sum Statement

The accumulator variable has the following characteristics:

- must be a numeric variable
- is automatically set to 0 before SAS reads the first observation
- is retained from one iteration to the next

The expression is defined with the following features:

- is any SAS expression
- is evaluated and the result added to the accumulator variable
- is ignored if missing

54

## Sum Statement

The sum statement is equivalent to using the RETAIN statement and the SUM function.

```
TotPopulation + Population2000;
```

```
retain TotPopulation 0;
TotPopulation =
    sum(TotPopulation, Population2000);
```

- The RETAIN statement causes a variable to retain its value from one iteration of the DATA step to the next and specifies an initial value for the variable.
- The SUM function returns the sum of the nonmissing arguments.

55

## RETAIN Statement

To initialize an accumulator variable to a value other than zero, include the accumulator variable in a RETAIN statement with an initial value.

|   | Population | Year |
|---|------------|------|
| 1 | 914950     | 1950 |
| 2 | 940841     | 1955 |
| 3 | 970689     | 1960 |
| 4 | 993236     | 1965 |
| 5 | 997357     | 1970 |
| 6 | 1062640    | 1975 |
| 7 | 1125027    | 1980 |
| 8 | 1163850    | 1985 |
| 9 | 1227928    | 1990 |

```
data FiveYearPop;
    set FiveYearPop;
    retain year 1945;
    year+5;
run;
```

58

## Reordering Data Set Variables

The RETAIN statement can be used in a DATA step to permanently change the order of the variables in an existing data set.

```
data annual_orders;  
  retain Customer_ID Month1-Month12;  
  set annual_orders;  
run;
```

The data set **annual\_orders** is used for input and output.



It is recommended that no additional processing be performed in the DATA step.